

Managing Claude Code — A Field Guide

The job isn't coding — it's supervising. Delegation & verification, mapped to the tools.

CORE MODEL: YOU'RE A MANAGER

Delegation is a **spectrum**. Good operators move up and down it on purpose — matched to how much domain expertise you have and how costly a wrong turn is.

- **1 · Pair-program** — you know the domain, review every line, course-correct constantly. Use when you're the expert.
- **2 · Delegate w/ spec** — you define *what* + *how it's checked*; Claude owns the *how*. Review outcomes, not keystrokes.
- **3 · Delegate w/ goal** — unfamiliar domain; you can only judge results. The "far-off manager" seat.

THE REAL SKILL: VERIFICATION

Discomfort in an unfamiliar domain isn't a workflow problem — it's a **verification** problem. Claude is very good at *sounding* right, so where you can't check the work by reading it, you're exposed.

When you can't judge the code, replace your judgment with **external verification you trust** — don't fake expertise, build instruments.

- **Ground truth first.** Judge by observable outcomes (territory %, respect, income in-game via viteburner) — not by whether the logic sounds correct.
- **Tests as trust anchor.** Have Claude write tests *first*; you vet the *expectations* (readable even when the impl isn't), then let code chase green.
- **Make it explain, then attack it.** "What would make this wrong? What did you assume?" — domain-ignorance bugs hide in unstated assumptions.

CAPABILITIES — WHEN THEY ACTUALLY HELP

- | | |
|----------------------|--|
| Plan Mode | The fix for unfamiliar-domain anxiety. Approve the <i>approach</i> before any code is written. |
| Subagents | Value is context hygiene , not parallelism — send an intern to the library, get back a summary. |
| Slash cmds | Codify your dev cycle so you stop re-typing prompts. (<code>.claude/commands</code>) |
| Hooks | Auto-run tests / lint / format on events. Removes supervisory busywork. |
| Skills | <code>/code-review</code> = cheap second opinion on code you can't fully vet yourself. |
| Worktrees | Enables <i>true</i> parallel work without file conflicts. File under "later." |
| Orchestration | Multi-agent workflows — for big audits/migrations. Overkill solo; know it exists. |

DO NEXT

- **1** Try **Plan Mode** on the next gang task — highest-leverage single change.
- **2** Shift verification onto **tests + in-game outcomes**, off your own reading.
- **3** Audit existing `.claude/commands` & `/skills` — you may own unused tools.
- **4** Ignore parallel agents until a **concrete** reason appears.

SKIP AS HYPE (FOR NOW)

Parallel agents as a status symbol · orchestration at solo scale · chasing "5 windows" before the single-window loop is second nature. Master one project well first — the rest is the parallel-reports mistake.